Programmation statistique automatisée

BUT Science des Données - Metz

Berhocoïrigoin Elena

Avant-propos

- N'hésitez pas à poser des questions avant, pendant, après

Objectifs

Python pour …

- Lecture de données
- Traitement de données
- Extraction d'informations (via statistiques, graphiques)
- Automatisation des traitements

Évaluation

- Un examen de 2 heures (dernière séance)
- Sur machine

Traitement des données en Python

- Pour pouvoir produire une **information de qualité**, il vous faut des **données de qualité** et dont vous avez une **bonne compréhension**

Traitement des données en Python

Client	Age
Georgette	NA
Anne	45
Paul	21
Louise	30
Pierrot	71
Bernadette	81
Annette	NA

Quel est l'âge moyen de mes clients ?

Problème : Beaucoup de femmes âgées et certains hommes âgées ne renseignent pas leurs âges par pudeur ou coquetterie

Valeurs manquantes porteuses de sens

Traitement des données en Python

Algorithme de recrutement Amazon, 2015-2017

Sexiste

Les données sont un reflet de notre société : secteur de la Tech, en majorité CV d'hommes, vocabulaire masculin

Le modèle a appris qu'être une femme était un inconvénient, critère d'exclusion

Or, le genre n'a aucun lien avec les compétences

Traitement des données en Python

Algorithme de recrutement Amazon, 2015-2017

Sexiste

Les données sont un reflet de notre société : secteur de la Tech, en majorité CV d'hommes, vocabulaire masculin

Le modèle a appris qu'être une femme était un inconvénient, critère d'exclusion

Or, le genre n'a aucun lien avec les compétences

Algorithmes de reconnaissance faciale, 2019

Racistes

189 logiciels, confondent plus fréquemment les visages non-blancs (asiatiques, afro-américains, amérindiens)

Images de visages caucasiens lors de l'entraînement, probablement reflet des biais des développeurs

Traitement des données en Python

- Pour pouvoir produire une information de qualité, il vous faut des données de qualité et dont vous avez une bonne compréhension
- Parmi les outils les plus utilisés : R (statistiques, biologie …) et Python (machine learning …)

Traitement des données en Python

- Pour pouvoir produire une information de qualité, il vous faut des données de qualité et dont vous avez une bonne compréhension
- Parmi les outils les plus utilisés : R (statistiques, biologie …) et Python (machine learning …)
- Pourquoi Python plutôt qu'un tableur (Excel …) ?
  - Automatiser des chaînes de traitement (ex: opérations répétitives)
  - Automatisation indépendante de la base de données
    - VBA spécifique à Excel, Excel payant
    - Code VBA lié à un fichier
    - Code VBA lié à des cellules, code facilement cassé par des changements dans l'organisation du fichier

Numpy

Numpy

Numpy est un package python

Module : **Fichier Python** contenant un **ensemble de fonctions** et de **variables globales**. Extension `.py`.

Package : **Répertoire** contenant des **collections de modules** et un fichier `__init__.py`, grâce auquel l'interpréteur interprète le répertoire comme un package. Un package peut également contenir des sous-packages.

Librairie : Définition moins claire. Sens généralement admis : ensemble de packages.

Numpy

Numpy est un package python

Boîte à outils qui regroupe un ensemble de fonctions dédiées à un objectif commun.

Les modules et les packages évitent d'avoir à coder soi-même chaque fonction.

Les fonctions disponibles via les modules et les packages sont généralement plus optimisées que ce que vous pourriez coder vous-même en Python. Notamment parce que certaines fonctions Python sont en réalité codées en C.

Numpy

- Numpy est la brique de base de l'écosystème Python dédié aux données
- L'objet principal en Numpy est l'**array**, un tableau multidimensionnel

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Unidimensionnel : vecteur

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Unidimensionnel : vecteur

$$\begin{array}{cc} & \text{Colonnes} \\ & \begin{array}{ccc} 0 & 1 & 2 \end{array} \\ \text{Lignes} & \begin{array}{c} 0 \\ 1 \\ 2 \end{array} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \end{array}$$

Bidimensionnel : matrice

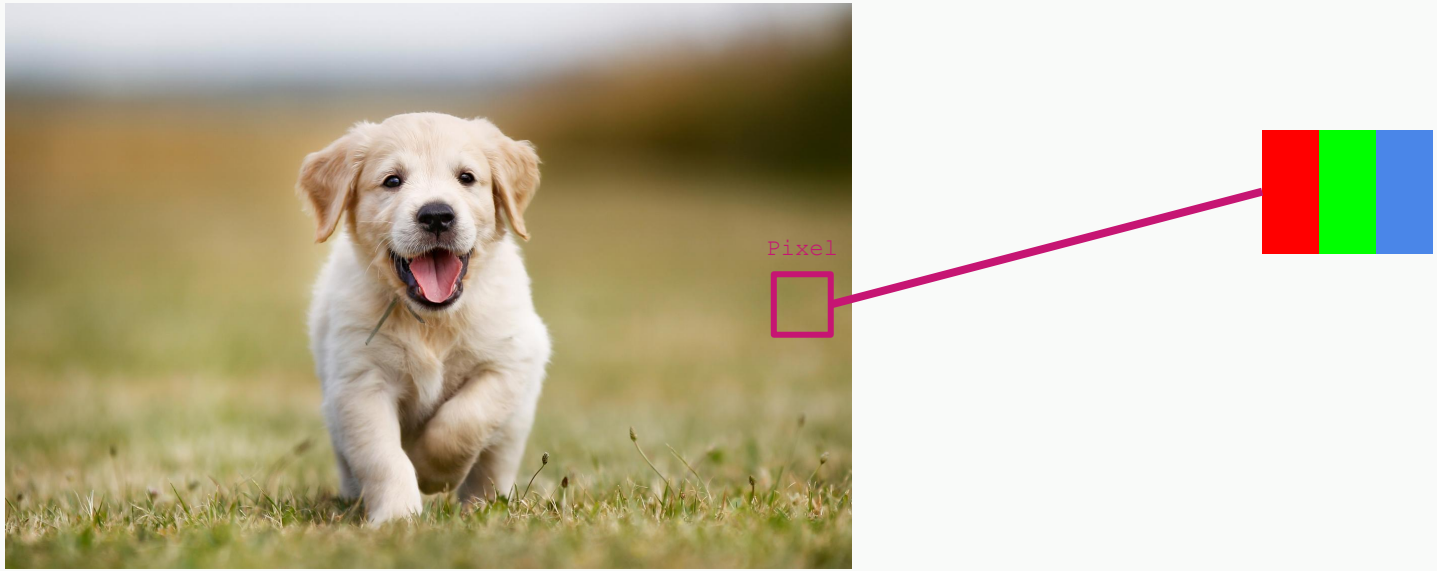


Image RGB (Red Green Blue)

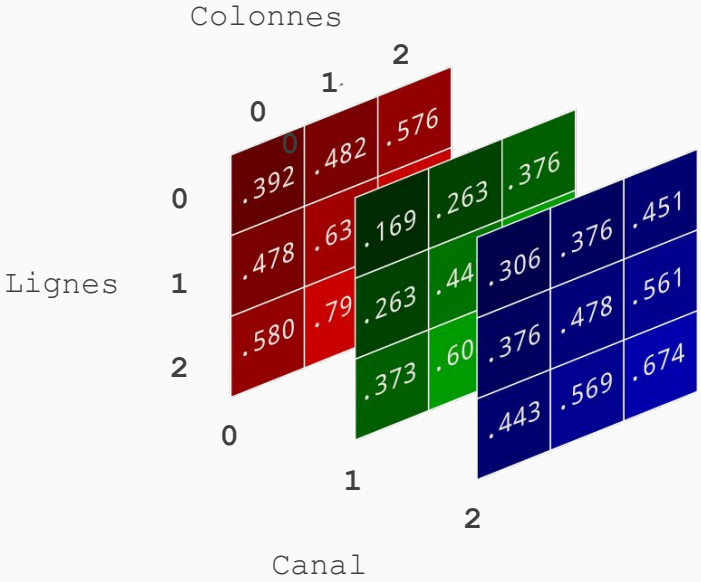
$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Unidimensionnel : vecteur

Colonnes

Lignes	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Bidimensionnel : matrice



Tridimensionnel
Exemple : image

Numpy

- Numpy est la brique de base de l'écosystème Python dédié aux données
- L'objet principal en Numpy est l'**array**, un tableau multidimensionnel
- Un array ne peut contenir qu'un seul type de données (contrairement aux listes)

Numpy

- Numpy est la brique de base de l'écosystème Python dédié aux données
- L'objet principal en Numpy est l'**array**, un tableau multidimensionnel
- Un array ne peut contenir qu'un seul type de données (contrairement aux listes)
- **Création** : liste, liste de listes, fonctions numpy

Numpy

- Numpy est la brique de base de l'écosystème Python dédié aux données
- L'objet principal en Numpy est l'**array**, un tableau multidimensionnel
- Un array ne peut contenir qu'un seul type de données (contrairement aux listes)
- **Création** : liste, liste de listes, fonctions numpy
- **Accéder** à un ou plusieurs éléments de Numpy :
  - Indexation classique
  - Slicing
  - Filtre logique

Indexation classique

Unidimensionnel :

```
arr[indice(s)]
```

Bidimensionnel :

```
arr[ligne(s),colonne(s)]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[ligne,colonne]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[1,1]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[lignes,colonnes]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],colonnes]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

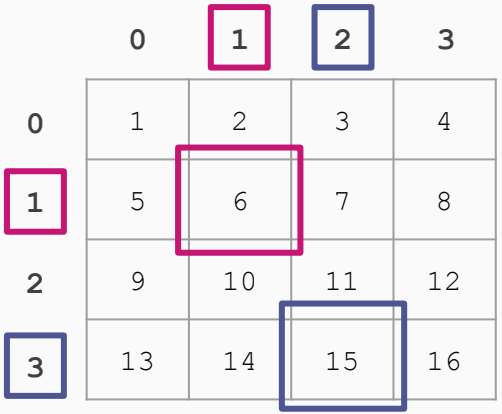
Slicing

Unidimensionnel :

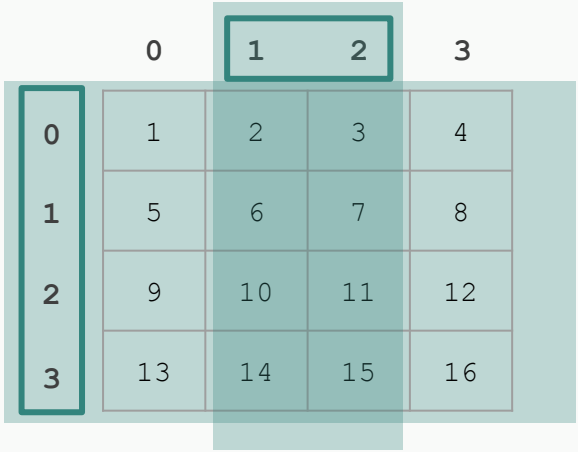
```
arr[start:stop:step]
```

Bidimensionnel :

```
arr[start:stop:step,start:stop:step]
```

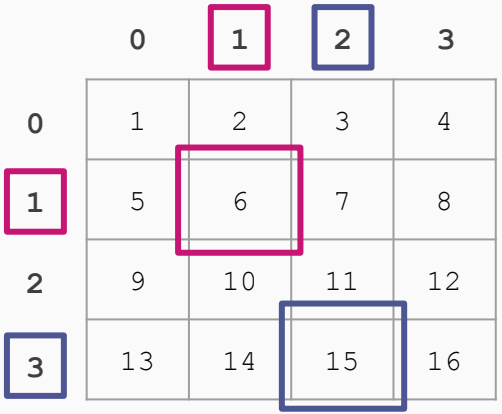


Indexation

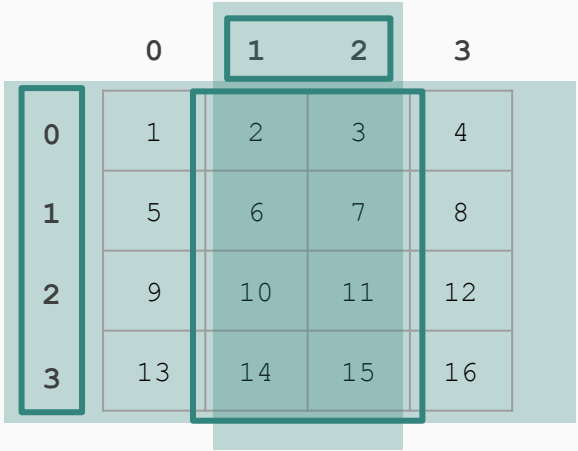


Slicing

```
arr[[1,3],[1,2]]
```



Indexation



Slicing

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[start:stop:pas,start:stop:pas]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[0:4:1,start:stop:pas]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[:,start:stop:pas]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[:,1:3:1]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

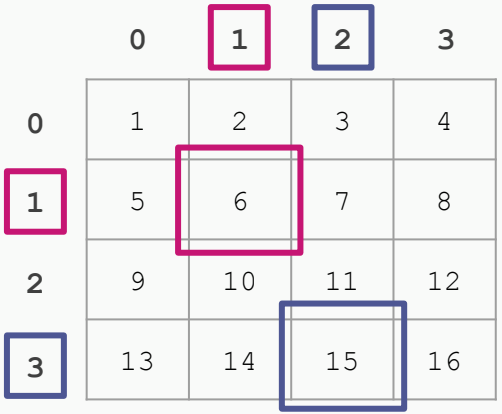
Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

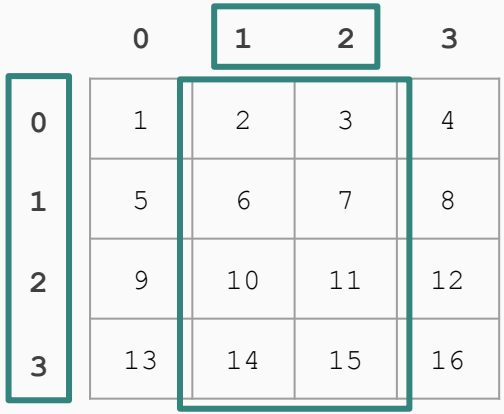
Slicing

```
arr[:,1:3]
```



Indexation

```
arr[[1,3],[1,2]]
```



Slicing

```
arr[:,1:3]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[start:stop:pas,start:stop:pas]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[2:4:1,start:stop:pas]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[2:4,start:stop:pas]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[2:4,0:3:2]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[2:4,:3:2]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Slicing

```
arr[2:4,:3:2]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[1,3],[1,2]]
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Indexation

```
arr[[2,2,3,3],[0,2,0,2]]
```

Numpy

- Numpy est la brique de base de l'écosystème Python dédié aux données
- L'objet principal en Numpy est l'**array**, un tableau multidimensionnel
- Un array ne peut contenir qu'un seul type de données (contrairement aux listes)
- **Création** : liste, liste de listes, fonctions numpy
- **Accéder** à un ou plusieurs éléments de Numpy :
  - Indexation classique
  - Slicing
  - Filtre logique

Numpy

- Numpy est la brique de base de l'écosystème Python dédié aux données
- L'objet principal en Numpy est l'**array**, un tableau multidimensionnel
- Un array ne peut contenir qu'un seul type de données (contrairement aux listes)
- **Création** : liste, liste de listes, fonctions numpy
- **Accéder** à un ou plusieurs éléments de Numpy :
  - Indexation classique
  - Slicing
  - Filtre logique
- **Modification** : sélection et affectation, fonctions numpy, combinaison de plusieurs arrays

Numpy

- Numpy est la brique de base de l'écosystème Python dédié aux données
- L'objet principal en Numpy est l'**array**, un tableau multidimensionnel
- Un array ne peut contenir qu'un seul type de données (contrairement aux listes)
- **Création** : liste, liste de listes, fonctions numpy
- **Accéder** à un ou plusieurs éléments de Numpy :
  - Indexation classique
  - Slicing
  - Filtre logique
- **Modification** : sélection et affectation, fonctions numpy, combinaison de plusieurs arrays
- **Agrégation** : fonction Numpy pour statistiques descriptives simples

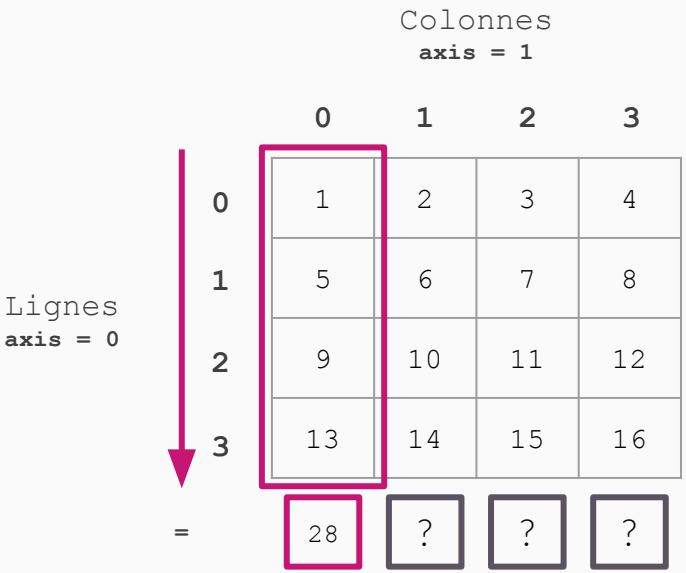
Somme des valeurs pour chaque colonne :

Colonnes
axis = 1

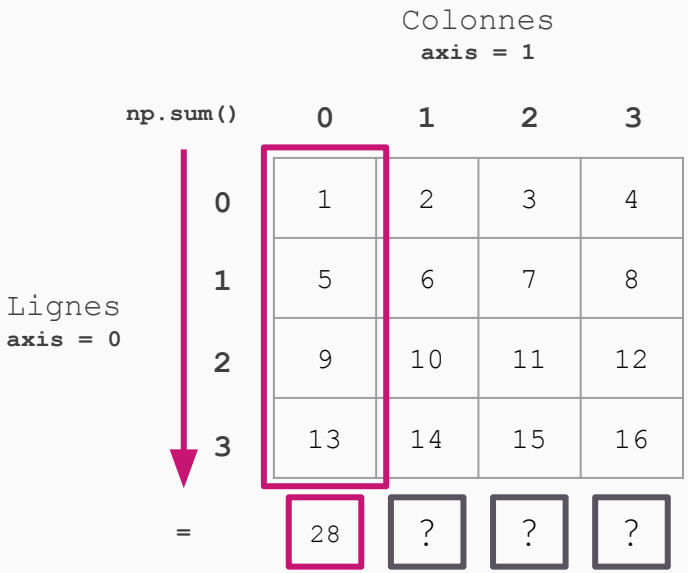
Lignes
axis = 0

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16
	?	?	?	?

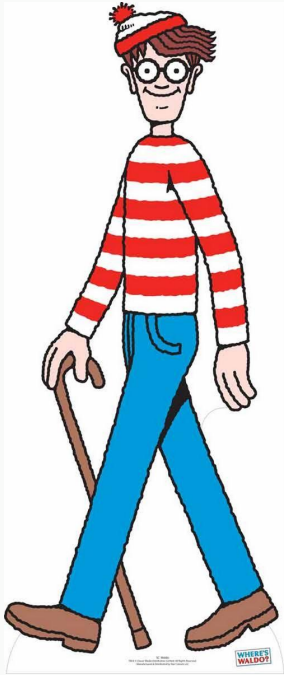
Somme des valeurs pour chaque colonne :



Somme des valeurs pour chaque colonne :




```
np.sum(arr, axis=0)
```


WHERE'S
WALDO?

Résumé des fonctions / méthodes vues

Création :

- `np.array()`
- `np.arange()`
- `np.linspace()`
- `np.zeros()`
- `np.ones()`
- `np.full()`
- `np.eye()`

Slicing :

- `<array>[start:stop:step]`
- `<array>[start:stop:step,start:stop:step ]`

Propriétés :

- `<array>.ndim`
- `<array>.size`
- `<array>.shape`
- `<array>.dtype`
- `len(<array>)`
- `type()`

Indexation classique :

- `<array>[indice(s)]`
- `<array>[ligne(s), colonne(s)]`

Filtre logique :

- `<array>[<expression>]`
- `&`
- `|`
- `~`
- `np.isnan()`
- `np.any()`
- `np.all()`
- `np.array_equal()`

Résumé des fonctions / méthodes vues

Modification :

- `<array>.astype()`
- `<array>.flatten()`
- `<array>.T` ou `np.transpose(<array>)`
- `np.insert()`
- `np.append()`
- `np.delete()`
- `np.sort()`
- `np.concatenate()`
- `+, x, -, /`
- `<array>[...] = ...`
- `<array> = ...`

Statistiques :

- `np.sum()`
- `np.mean()`
- `np.min()`
- `np.argmin()`
- `np.max()`
- `np.argmax()`
- `np.std()`
- `np.median()`
- `np.where()`

MERCI